Drone Position Estimation using Batch and Recursive Least Squares

Piyush S. More

1 Problem Description

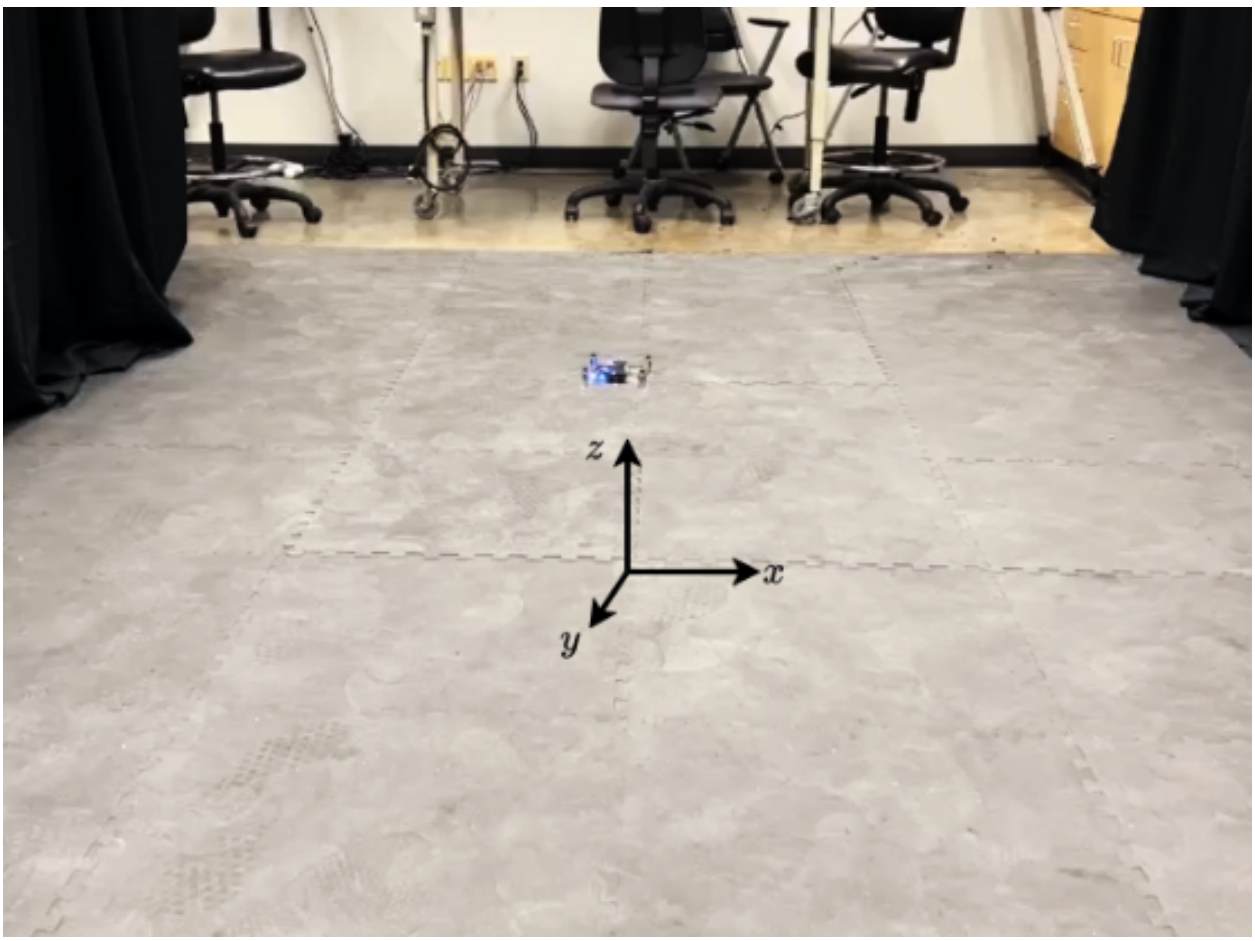

Figure 1: Experimental setup showing the Crazyflie drone and global coordinate frame used for position estimation experiments.

A Crazyflie drone is hovering at a constant position while measurements are collected using two sensors.

- Sensor S_1 provides full position measurements (x, y, z).

- Sensor S_2 provides height measurements only (z).

A total of 200 synchronized measurements are available from each sensor. The measurement noises follow Gaussian distributions:

$$\sigma_x^{S_1} = 0.5\,m$$

$$\sigma_y^{S_1} = 0.5\,m$$

$$\sigma_z^{S_1} = 0.5\,m$$

$$\sigma_z^{S_2} = 1\,m$$

All measurements are independent.

The goal is to estimate the constant drone position

$$\theta = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

using both Batch Least Squares and Recursive Least Squares estimators.

2 Batch Least Squares Estimation

The measurement model can be written as

$$y_k = H\theta + v_k$$

where the measurement matrix is

$$H = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

and the measurement vector is

$$y_k = \begin{bmatrix} x_{S_1} \\ y_{S_1} \\ z_{S_1} \\ z_{S_2} \end{bmatrix}.$$

The Batch Least Squares estimator is given by

$$\hat{\theta} = (H^T R^{-1} H)^{-1} H^T R^{-1} y$$

where R is the measurement covariance matrix.

2.1 Estimated Drone Position

The estimated position obtained from the batch least squares estimator is

$$\hat{\theta} = \begin{bmatrix} 0.0257 \\ -0.0363 \\ 0.7731 \end{bmatrix}$$

3 Estimator Covariance

The covariance matrix of the Batch Least Squares estimator is

$$P = \begin{bmatrix} 0.0025 & 0 & 0 \\ 0 & 0.0025 & 0 \\ 0 & 0 & 0.0017 \end{bmatrix}$$

The diagonal elements of P represent the variances of the estimated states x, y, and z.

Although sensor S_2 has a higher noise variance than sensor S_1, including it improves the estimation of the height coordinate because it provides additional information about z.

Therefore, incorporating sensor S_2 improves the overall estimation accuracy.

4 Recursive Least Squares Estimation

Recursive Least Squares (RLS) allows sequential updating of the estimate as new measurements arrive.

The update equations are

$$K_k = P_{k-1} H^T (H P_{k-1} H^T + R)^{-1}$$

$$\hat{\theta}_k = \hat{\theta}_{k-1} + K_k (y_k - H \hat{\theta}_{k-1})$$

$$P_k = (I - K_k H) P_{k-1}$$

The recursive estimator is initialized with

$$\hat{\theta}(0) = \begin{bmatrix} 0 \\ 0 \\ 0.7 \end{bmatrix}$$

$$P(0) = I$$

The recursive estimator converges to the batch least squares estimate as more measurements are processed.

5 Estimation Results

The recursive estimator produces estimates of $(\hat{x}, \hat{y}, \hat{z})$ at each time step.

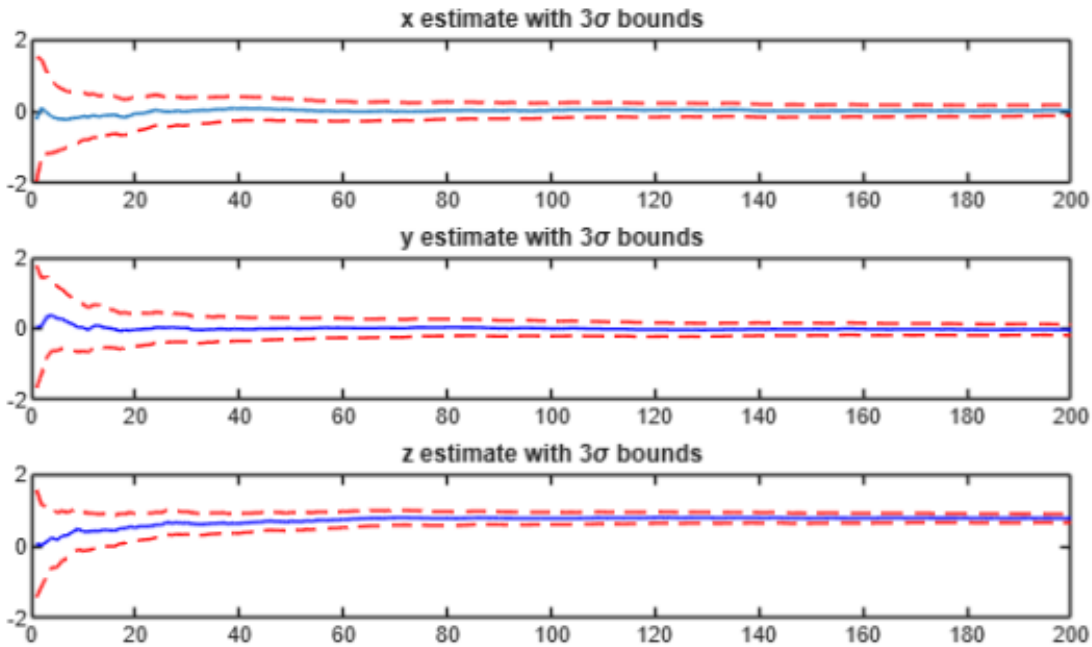

Figure 2: Recursive Least Squares estimates of the drone position $(\hat{x}, \hat{y}, \hat{z})$ with 3σ confidence bounds as a function of time step.

The dashed lines represent the 3σ confidence bounds of the estimator.

The distance between the boundary lines corresponds to

$$6\sigma$$

which represents the uncertainty range of the estimate.

6 Computation Time Comparison

The computation time of both algorithms was measured using the MATLAB commands `tic` and `toc`.

The measured computation times are

$$t_{Batch} = 0.003567 \text{ s}$$

$$t_{RLS} = 0.002707 \text{ s}$$

Batch Least Squares processes all measurements simultaneously and requires inversion of large matrices. In contrast, Recursive Least Squares updates the estimate sequentially as new measurements arrive and avoids recomputing matrix inversions for the entire dataset.

Although both methods are fast for this dataset, Recursive Least Squares is generally more suitable for real-time estimation because it updates the solution incrementally as new measurements become available.

7 Conclusion

This study demonstrated the estimation of a drone's position using noisy sensor measurements. Both Batch Least Squares and Recursive Least Squares methods were implemented.

The recursive estimator converges to the batch solution while allowing real-time processing and improved computational efficiency.